

Git & GitHub 협업 및 연동

5. GitHub 연동 및 원격 저장소 설정 (Remote)

5.1. GitHub 저장소 생성 (github 관리자만 확인)

GitHub Web UI 를 통해 원격 저장소(Repository)를 생성합니다.

The screenshot shows the GitHub 'New repository' page. At the top, there's a 'Dashboard' header with a search bar. Below it, the 'Top repositories' section has a 'Find a repository...' input field and a 'New' button (circled in red). The 'Home' section is also visible. The 'New repository' form is divided into two sections: 'General' and 'Configuration'. In the 'General' section, the 'Repository name' field is circled in red, and the 'Description' field is also circled in red. In the 'Configuration' section, the 'Choose visibility' dropdown is set to 'Public' and is circled in red. At the bottom, there's a 'Create repository' button.

1. **Repository Name:** 프로젝트 이름 지정
2. **Public/Private:** 저장소 공개 범위 설정 (팀 내부용이면 Private 권장)
3. **Initialize this repository with:** .gitignore 파일은 프로젝트 언어/환경에 맞춰 미리 추가하는 것이 좋습니다.

5.2. 로컬 저장소 연결 (git remote add → 웬만하면 관리자만 하고 팀원들은 git clone 권장)(터미널에서 수행 후 github 사이트로 확인)

로컬에서 git init 으로 생성한 저장소에 GitHub 주소를 연결하여 원격 저장소를 지정합니다. (일반적으로 원격 저장소의 이름은 **origin**으로 지정합니다.)

#먼저 현재 브랜치 이름을 master 대신 main 으로 변경합니다(주로 main 으로 많이 씀) (-M 명령어가 현재 브랜치 명을 바꾼다는 의미입니다)

```
$ git branch -M main
```

#현재 branch 를 확인하고 싶다면 git branch 명령어를 사용하세요.

```
$ git branch
```

형식: git remote add <원격 이름> <GitHub 저장소 URL> 으로 로컬 저장소에 원격 저장소(github) 주소를 origin 이라는 이름으로 등록하여 연결합니다.

```
$ git remote add origin <git@github.com:~사용자명/~레포지토리명.git>
```

로컬의 main 브랜치 내용을 원격(origin)으로 최초 푸시하고, 이후 간편한 Push/Pull 을 위해 두 브랜치 간의 추적 관계를 설정합니다.

```
$ git push -u origin main
```

등록된 원격 저장소 목록 및 주소 확인

```
$ git remote -v
```

참고: git clone 명령어를 사용하여 저장소를 복제하면 이 **git remote add origin** 과정은 **자동으로 처리**됩니다. (5.8. 참고) → 팀원들 입장에서는 git clone 권장

5.3. 초기 코드 푸시 (git push -u)

로컬에서 작업한 코드를 GitHub 원격 저장소로 최초 전송하고, **추적(Upstream)** 설정을 합니다.

로컬 main 브랜치를 원격 origin 에 올리고(-u), 이후 push/pull 시 간결하게 사용 가능하도록 추적 설정

```
$ git push -u origin main
```

6. 보안 및 인증 설정 (SSH Key)

원격 저장소에 Push/Pull 할 때마다 사용자 이름과 비밀번호를 반복해서 입력해야 하는 불편함을 해소하고 보안을 강화하기 위해 **SSH (Secure Shell) Key** 를 설정합니다.

6.1. Git 인증 방식 이해

- **HTTPS:** 아이디/비밀번호 또는 PAT(Personal Access Token)를 사용합니다. 비교적 설정은 쉽지만, 매번 인증 정보를 입력해야 하는 번거로움이 있습니다.
- **SSH:** 공개키/비밀키 쌍을 사용합니다. 최초 설정은 필요하지만, 이후 비밀번호 입력 없이 편리하게 사용 가능합니다. **협업 시 권장되는 방식**입니다.

6.2. SSH Key 생성 및 등록 (개인 작업)

SSH 키는 사용자 **개인**의 컴퓨터에서 생성하고 GitHub 계정에 등록합니다.

1. **SSH Key 생성:** (일반적으로 ~/.ssh/ 디렉토리에 저장됩니다.)
키 타입을 ed25519 로 생성하고, 이메일 주소를 코멘트로 남깁니다.
\$ ssh-keygen -t ed25519 -C "your_email@example.com"
2. **공개키 내용 확인:** 생성된 공개키 파일(id_ed25519.pub)의 내용을 터미널에 출력합니다.
\$ cat ~/.ssh/id_ed25519.pub
3. **GitHub 등록:**
 - GitHub 접속 → **Settings** → **SSH and GPG keys** 이동
 - **New SSH key** 버튼 클릭 후, Key 입력란에 위 명령어 결과로 나온 **공개키 전체 내용**을 복사해 붙여넣습니다.

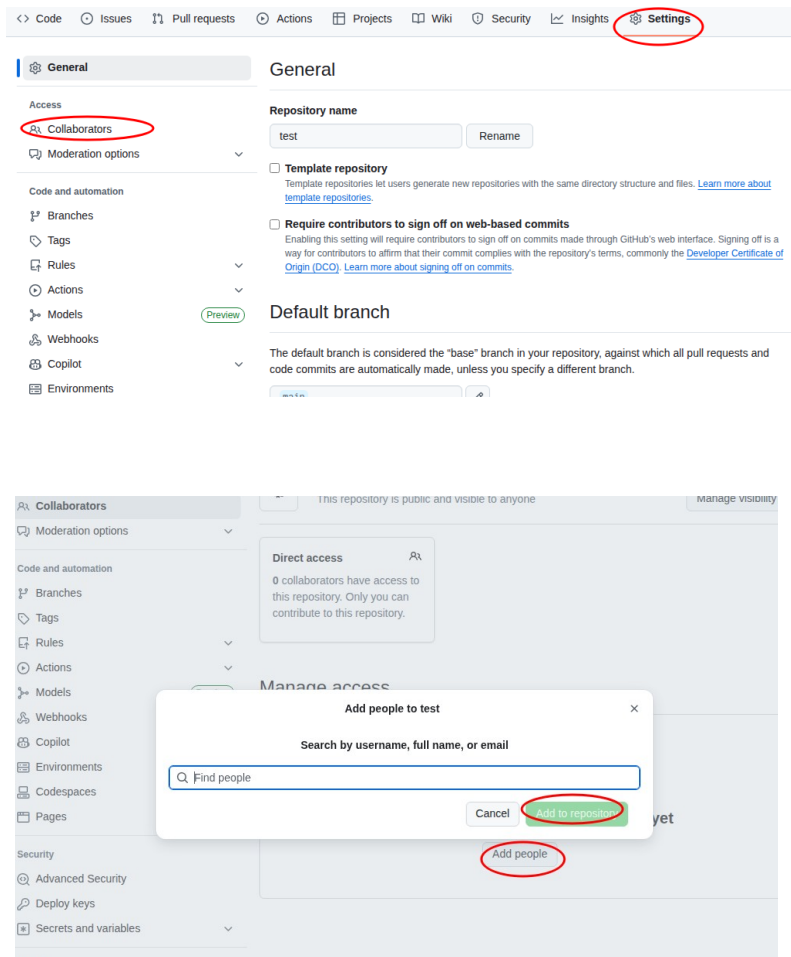
6.3. 키 연결 테스트

연결 테스트. 성공 시 'You've successfully authenticated' 메시지가 출력됩니다.

\$ ssh -T git@github.com

7. 팀원 협업 환경 설정 및 시작

7.1. 팀원 초대 및 권한 부여 (관리자 역할)



팀원을 초대하면 기본적으로 다음과 같은 권한 등급 중 하나가 부여됩니다.

권한 등급 (Role)	의미	추천 대상
Read (읽기)	저장소 코드를 볼 수만 있습니다. Push나 Merge는 불가능합니다.	문서 작성자, PM, 외부 감사자
Triage (분류)	이슈(Issue)와 Pull Request를 관리(분류, 레이블 지정 등)할 수 있습니다. 코드는 수정 불가.	프로젝트 관리자
Write (쓰기)	코드를 Push 할 수 있고, 브랜치를 만들거나 병합할 수 있습니다.	일반적인 팀 개발자
Maintain (유지 보수)	저장소 설정 변경 및 다른 Collaborator를 초대/제거할 수 있습니다.	서브 리더, 팀 리더
Admin (관리자)	모든 권한(저장소 삭제 포함)을 가집니다.	프로젝트 소유자

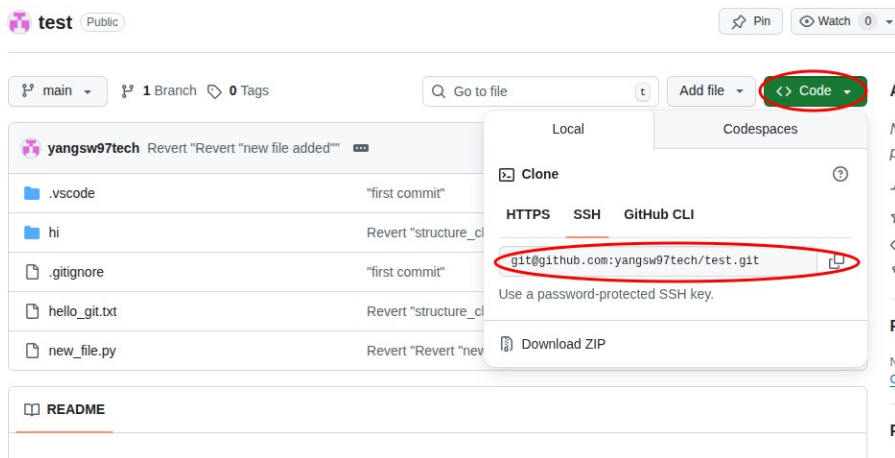
관리자는 GitHub 웹사이트를 통해 팀원들을 저장소에 초대합니다.

1. GitHub 저장소 접속 → **Settings** 탭 → **access** 메뉴 이동
2. **collaborators** 를 클릭하여 팀원의 GitHub ID 또는 이메일을 검색하여 초대합니다(add people).
3. **권한 설정**: 초대할 때 일반적으로 개발자는 **Write (쓰기)** 권한을 부여받아야 Push 할 수 있습니다.

7.2. 팀원의 작업 시작 (git clone)

초대받은 팀원은 git clone 명령으로 저장소를 복제하여 작업을 시작합니다.

- git clone 은 ****git init****과 ****git remote add origin****을 한 번에 처리해 주며, 저장소의 모든 이력을 로컬로 가져옵니다. 깃허브 사이트의 code 부분을 클릭하여 주소를 복사하고 git clone <주소> 형태로 사용하면 됩니다.



인증 방식	명령어 (권장)
SSH (권장)	\$ git clone git@github.com:user/repo.git
HTTPS	\$ git clone https://github.com/user/repo.git

복제 후 확인:

clone 이 완료된 폴더로 이동하여 확인

\$ git remote -v

이미 'origin'이라는 이름으로 원격 저장소 연결이 자동 설정되어 있음을 확인

8. 협업의 기본: Pull & Push

8.0. 각자의 branch 파기

팀원들은 main 에서 뺏어나와 각자의 작업을 각자의 branch 에서 수행합니다. Git 에서 독립적인 작업 공간을 만드는 기능으로써 안정적인 분업 개발을 가능케 합니다.

팀원들은 저장소를 `git clone` 하여 로컬 저장소를 만든 후, 다음 단계를 통해 각자의 작업 브랜치를 생성합니다.

순서	명령어	역할
1 단계: 현재 브랜치 확인	<code>\$ git branch</code>	로컬 저장소에 존재하는 브랜치 목록과 현재 위치를 확인합니다.
2 단계: 최신 main 동기화	<code>\$ git pull origin main</code>	혹시 모를 변경 사항에 대비하여 main 브랜치를 원격 최신 상태로 업데이트합니다.
3 단계: 브랜치 생성 및 이동	<code>\$ git switch -c <새 브랜치 이름></code>	새로운 브랜치를 생성함과 동시에 그 브랜치로 이동하여 작업을 시작합니다.
브랜치 이동	<code>\$ git switch <브랜치 이름></code>	언제든지 현재 작업 브랜치를 변경할 수 있습니다.

8.1. github 의 최신 내용 가져오기(git Pull)

팀원들의 최신 변경 사항을 로컬로 가져와 병합(Merge)합니다.

- **git fetch**: 원격 저장소의 최신 이력(커밋)을 로컬로 가져오지만, **로컬 작업 브랜치에는 병합하지 않습니다.** (단순 확인용 → 근데 이 방식은 `git log` 에도 안뜨고 확인하기 어려우니 그냥 `pull` 쓰는게 좋을듯)
- **git pull**: `git fetch` 와 `git merge` 를 한 번에 수행하여, 최신 이력을 가져와 **현재 브랜치에 즉시 병합**합니다.

메인 브랜치로 이동

```
$ git switch main
```

원격의 최신 내용을 가져와 현재 작업 중인 브랜치에 병합 (가장 일반적인 사용)

```
$ git pull origin main
```

자신 고유의 작업 브랜치로 이동

```
$ git switch <개인 브랜치명>
```

```
# 최신 상태의 로컬 main 브랜치의 내용을 내 개인 브랜치로 병합
```

```
$ git merge main
```

8.1.1. git merge 세부 옵션

기본 사용법은 현재 병합받을 브랜치(타겟 브랜치)로 이동하고 git merge 로 병합할 브랜치(소스 브랜치)로 병합하는 것입니다.

```
$ git switch <trg branch>
```

```
$ git merge <src branch>
```

No fast forward : **3-way Merge** 를 강제하는 옵션입니다. 브랜치 히스토리가 일직선으로 이어지더라도 **반드시 병합 커밋을 생성**하여 브랜치 분기/통합 이력을 남깁니다

```
$ git merge --no-ff <소스 브랜치 이름>
```

squash 압축 병합 : 소스 브랜치(feature/X)에서 발생한 ****모든 커밋(C1, C2, C3...)****을 하나로 **압축**하여 타겟 브랜치(main)에 **단일 커밋**으로 적용합니다.

```
$ git merge --squash <소스 브랜치 이름>
```

```
$ git commit -m "Feat: 새로운 기능 통합 (Squashed from feature/X)"
```

병합 취소: 병합(git merge) 중에 ****충돌(Conflict)****이 발생했을 때, 병합 작업을 중단하고 **병합을 시작하기 전 상태로** 되돌아갑니다. 충돌이 복잡하여 나중에 다시 해결하고 싶을 때 사용합니다.

```
$ git merge --abort
```

8.2. 로컬 변경 사항 원격 반영 (git push)

로컬에서 커밋된 내용(Commit History)을 원격 저장소에 업로드합니다.

로컬의 변경 사항을 원격 저장소의 origin 에 있는 개인 브랜치로 푸시
\$ git push origin <개인 브랜치명>

주의: Push 전에 반드시 ****git pull****을 먼저 실행하여 원격 저장소의 최신 내용을 반영하고 **충돌(Conflict)** 여부를 확인해야 합니다.

8.3. 충돌 (Conflict) 해결

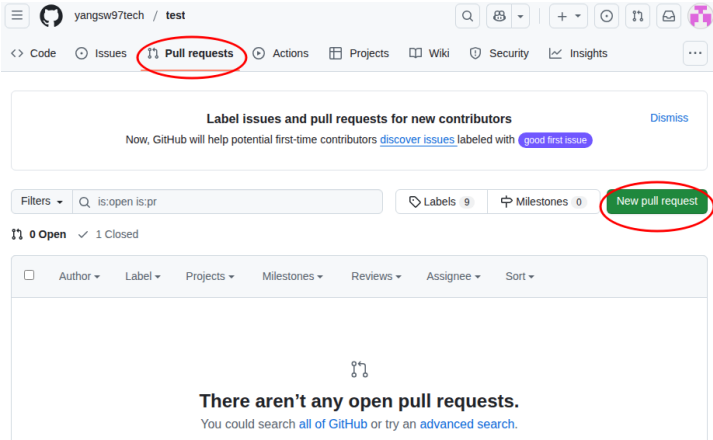
서로 다른 파일의 **같은 라인**을 수정하고 각자 Push 하려 할 때 충돌이 발생합니다.

1. **git pull 시 충돌 발생:** Git 이 파일을 자동으로 병합하지 못하고 충돌 상태에 진입합니다. Git 은 충돌된 파일 내부에 <<<<<<, =====, >>>>>> 같은 ****마커(Marker)****를 추가합니다.
2. **충돌 해결 (수동 작업):** 충돌이 발생한 파일을 열어 이 마커들을 보고 **남겨야 할 최종 코드만** 수동으로 선택하여 정리합니다.
3. **해결된 파일 스테이징:** 충돌 마커를 모두 제거한 후, 해결이 완료된 파일을 다시 스테이징합니다.
\$ git add <충돌 파일명>
4. **병합 커밋:** Git 이 자동으로 생성하는 병합 커밋 메시지를 확인하고 커밋합니다.
\$ git commit
5. **Push:** 충돌이 해결된 병합 커밋을 원격 저장소에 Push 합니다.
\$ git push

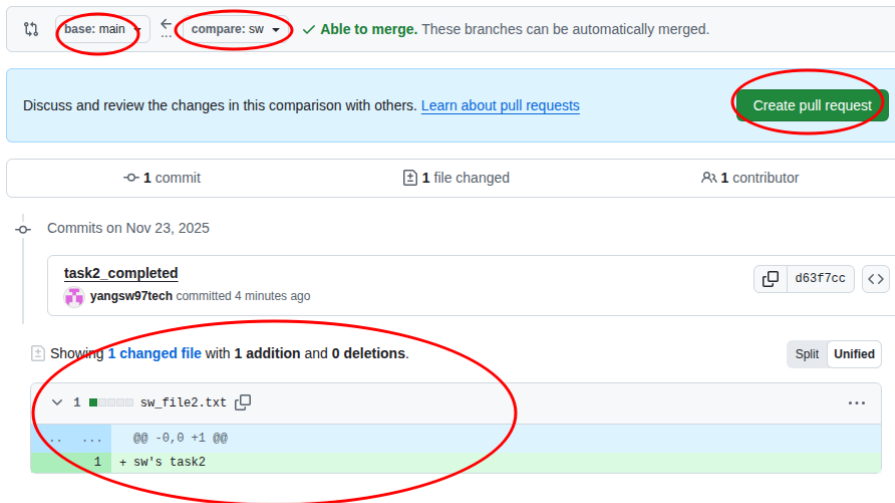
8.4. pull request

본인의 branch 에서 작업하다가 이제 어느정도 기능이 구현되면 main branch 에 합쳐야 합니다.

이때 팀원들은 github 에서 pull request 를 이용하여 main 으로의 병합을 요청합니다.

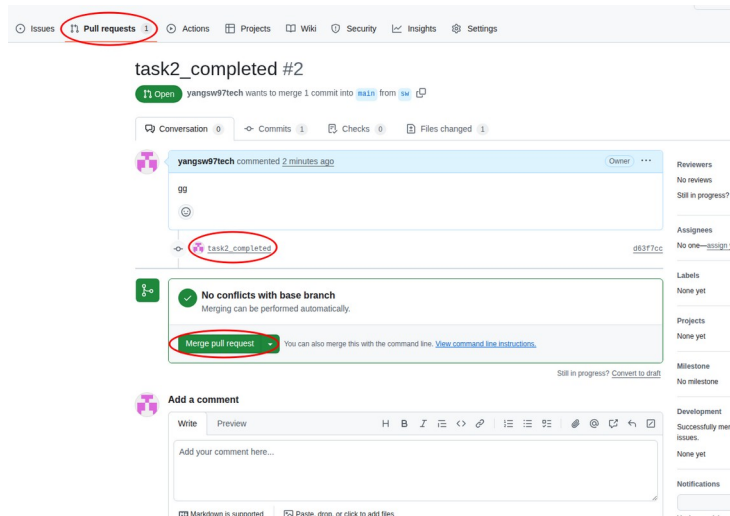


위와 같이 github 사이트로 들어가서 pull request 탭의 new pull request 를 클릭합니다.



최상단의 able to merge 옆의 두 항목으로 어떤 branch 내용(개인 브랜치)을 어떤 곳(main branch)으로 병합하려 하는지 설정합니다. 아래서 변경사항 또는 commit 등을 체크한 뒤 create pull request 를 통해 병합을 요청합니다. Add a title 이나 Add a description 으로 변경사항을 간단하게 정리하고 최종 create pull request 버튼을 누르면 됩니다.

8.4.1.관리자 merge pull request



관리자는 pull request 를 검토하고 merge 하여 main 브랜치에 병합할 수 있습니다.

Merge 후에는 관리자는 팀원들에게 병합 사실을 보고하고 팀원들은 자신의 로컬 main 브랜치를 최신 상태로 동기화 합니다.

1. main 브랜치로 이동

```
$ git switch main
```

2. 원격의 최신 내용을 가져와 병합

```
$ git pull origin main
```

8.5. 작업 브랜치 삭제

병합이 완료되어 필요가 없어진 브랜치를 삭제할 수 있습니다. 보통 개인 브랜치에서 또 분기를 나누어 기능단위로 브랜치를 생성 하였을 시, 해당 기능이 병합되고 나면 기능단위 브랜치를 삭제합니다. (깔끔하게 관리하려면 삭제해도 되지만 불안하면 놔두어도 될 것 같음)

로컬에서 다른 삭제하려는 브랜치가 아닌 다른 브랜치로 이동하고 삭제

```
$ git switch main
```

```
$ git branch -d feature/login
```

원격에서도 해당 브랜치 제거 요청

```
$ git push origin --delete feature/login
```

GitHub Main 브랜치 보호 규칙 설정 가이드

목표: 팀원들이 main 브랜치에 직접 코드를 Push 하는 것을 막고, 반드시 Pull Request(PR)와 코드 리뷰를 통해서만 병합되도록 하여 프로젝트의 안정성을 유지합니다.

주의: 이 설정은 저장소에 대한 **관리자(Admin) 권한**을 가진 사용자만 수행할 수 있습니다.

1. 보호 규칙 설정 경로

1. GitHub 웹사이트에서 해당 프로젝트 ****저장소(Repository)****로 이동합니다.
2. 상단 탭 목록에서 ****Settings****를 클릭합니다.
3. 왼쪽 사이드바 메뉴에서 ****Branches****를 클릭합니다.
4. Add branch rule 버튼을 클릭합니다.
5. rule name 을 작성합니다.(규칙 이름)

2. 보호 규칙 대상 지정

규칙을 적용할 **대상 브랜치**를 지정합니다. 일반적으로는 핵심 브랜치인 main 을 지정합니다.

입력 필드	입력 내용	설명
Target branches의 Add target	include by pattern 으로	보호 규칙을 main 브랜치에만 적용하도록 지정합니다. (브랜치 이름이 master 인 경우 master 를 입력)
	main	위와 같이 정확한 이름으로 제한하거나, release/* 등을 통해 패턴 매칭되는 모든 브랜치를 보호할 수도 있습니다.
	지정	

3. 핵심 보호 옵션 활성화

주요 목표인 "PR 과 리뷰를 통한 병합"을 강제하기 위해 다음 항목들을 활성화해야 합니다.

3.1. Pull Request 및 코드 리뷰 필수화 (핵심)

이 옵션은 main 브랜치에 코드를 통합하는 유일한 방법이 **Pull Request** 임을 강제합니다.

- Require a pull request before merging (병합 전 Pull Request 필수)
 - **활성화**
 - **효과:** PR 없이 main 브랜치로 직접 git push 하는 것을 차단합니다.
- Require approvals: (위 옵션 활성화 시 함께 나타남)

- **활성화 (1 이상)**
- **효과:** PR 이 승인(Approve)되려면 최소 **지정된 수**의 다른 팀원의 리뷰 승인을 받아야만 병합(Merge) 버튼이 활성화됩니다. (일반적으로 1 명을 설정하여 코드 품질을 보장)

3.2. 직접 푸시 및 강제 푸시 차단

이 옵션들은 main 브랜치를 오직 PR 을 통한 안전한 경로로만 변경할 수 있도록 합니다.

- Block force pushes (강제 푸시 차단)
 - **활성화**
 - **효과:** git reset --hard 후 git push --force 와 같이 **커밋 히스토리를 덮어쓰는** 위험한 행위를 관리자를 포함하여 모두에게 차단합니다.
- Restrict updates & creations (업데이트 제한)
 - **활성화**
 - **효과:** 권한이 있는 사용자라도 PR 절차를 거치지 않은 ****모든 직접 푸시(Update)****를 차단합니다. (PR 필수 옵션과 시너지를 냅니다.)

4. 규칙 생성 완료

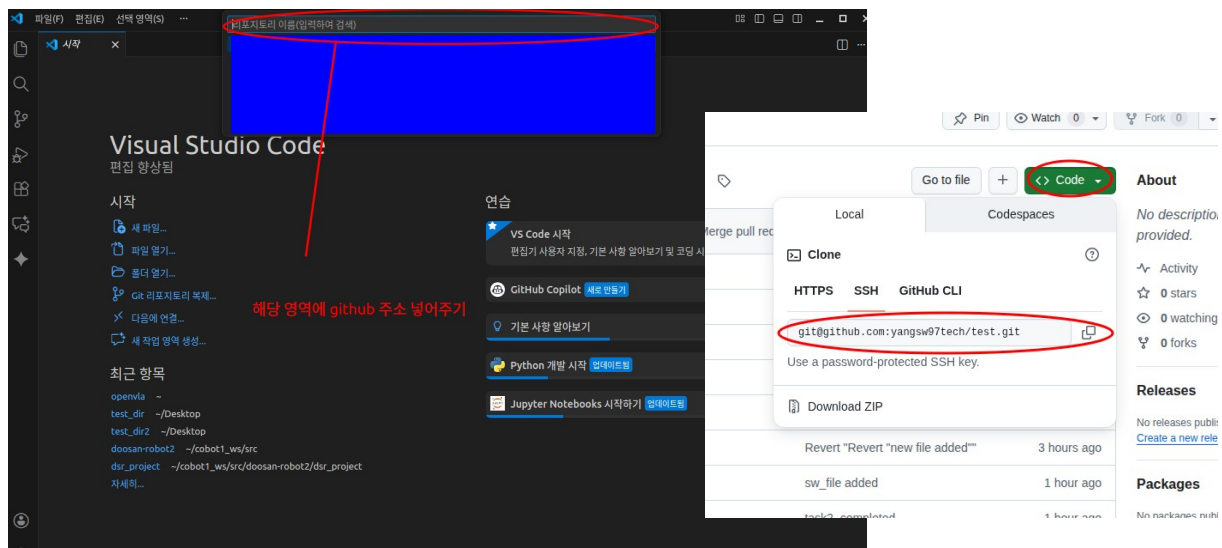
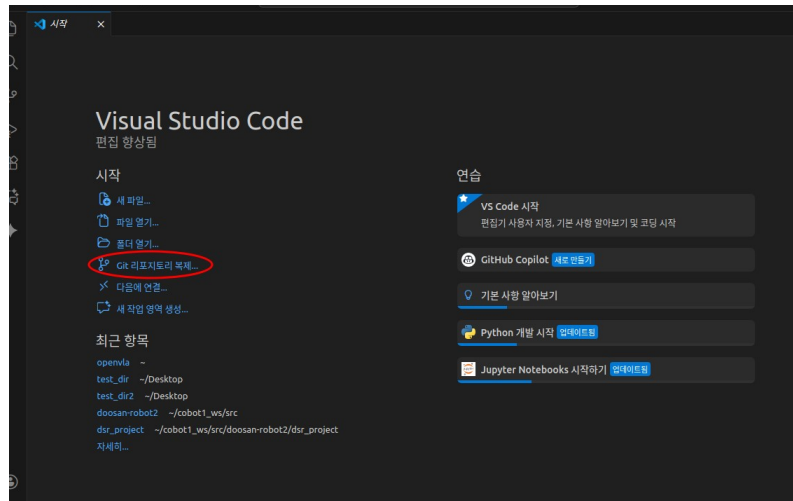
원하는 모든 옵션을 설정한 후, 페이지 하단의 Create 버튼을 클릭하여 규칙을 저장합니다.

이후부터 팀원들이 main 브랜치에 직접 푸시를 시도하면 **"보호 규칙 위반"** 오류 메시지를 받게 되며, 반드시 **작업 브랜치 → PR 생성 → 코드 리뷰 → Merge** 과정을 거쳐야만 main 브랜치에 코드가 반영됩니다.

8. VS Code 로 쉽게 협업

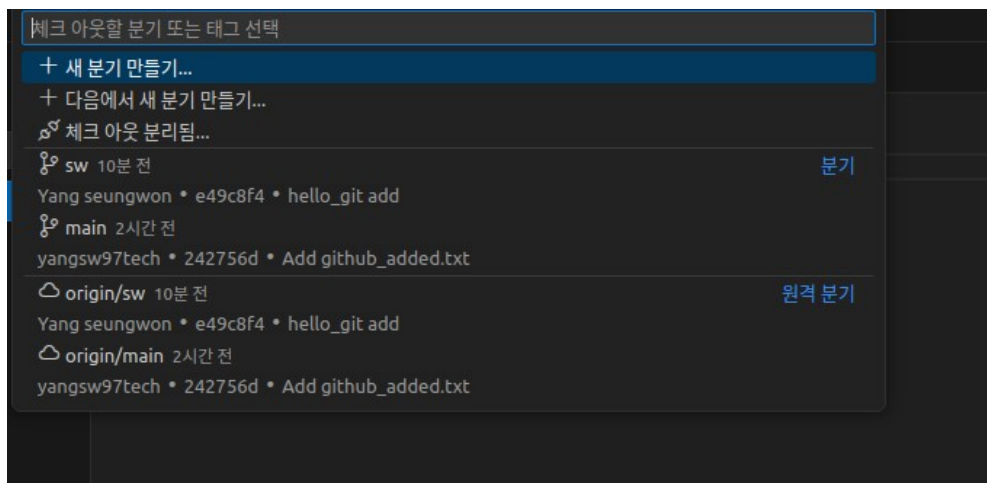
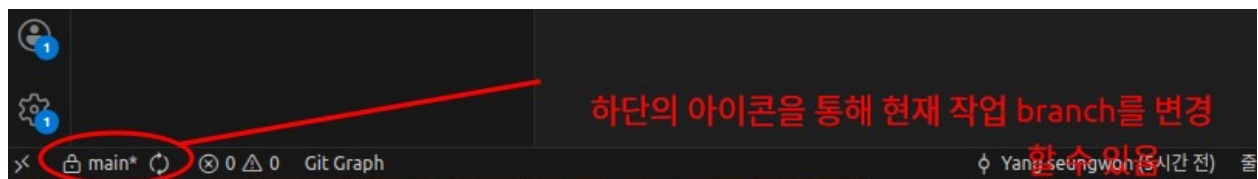
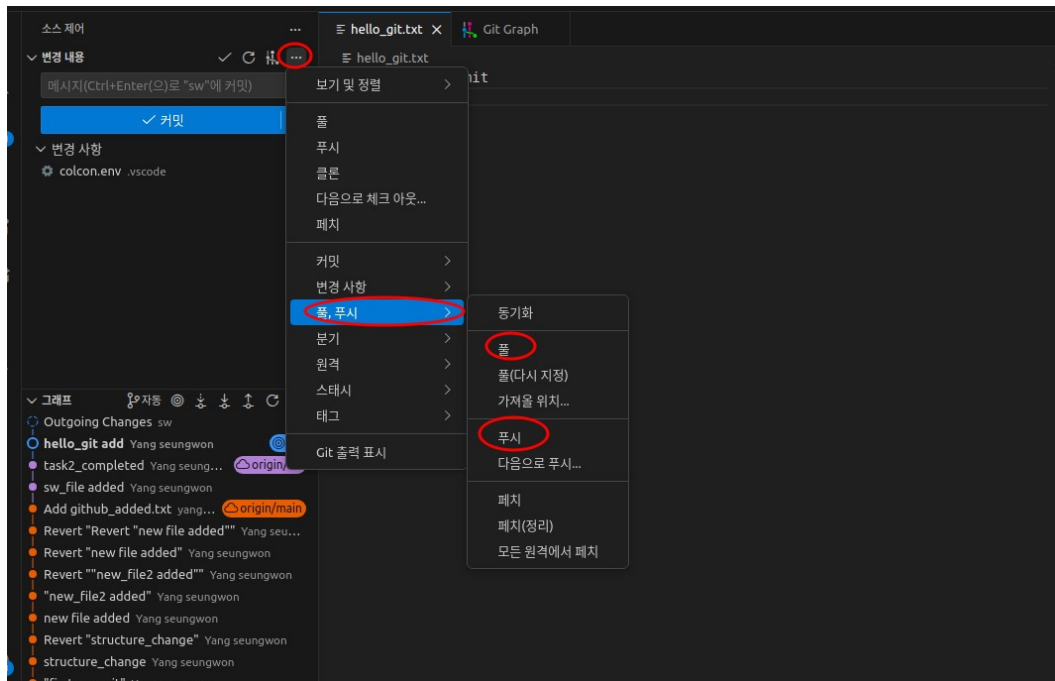
시작 화면으로 진입하였다면 welcome page 에서 clone git repository 버튼을 눌러 git clone 을 수행합니다.

만약 명령 팔레트로 git clone 을 하고자 한다면 Ctrl +Shift + P 를 눌러 명령 팔레트를 열고 Git: Clone 을 입력한 후 clone 하려는 github 주소를 입력합니다.

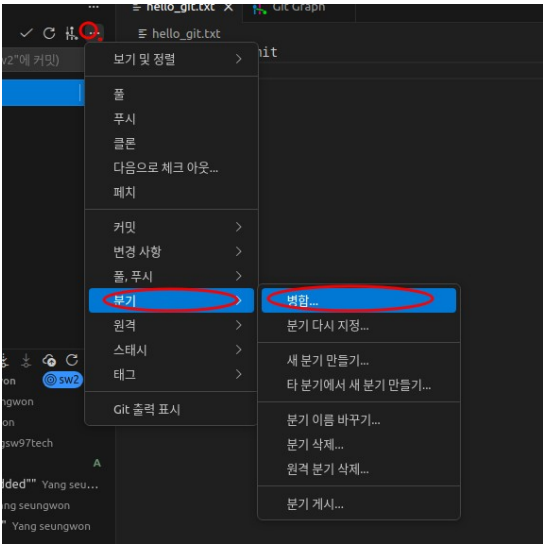


Git clone 과정을 통해 git init 과 git remote add origin 이 모두 자동 설정됩니다. 팀원은 이제 바로 작업을 시작할 수 있습니다.

어떤 파일을 staging 과 commit 까지 완료했다고 가정했을 때, 소스제어 탭에서 아래와 같은 버튼으로 pull push 를 진행 할 수 있습니다.



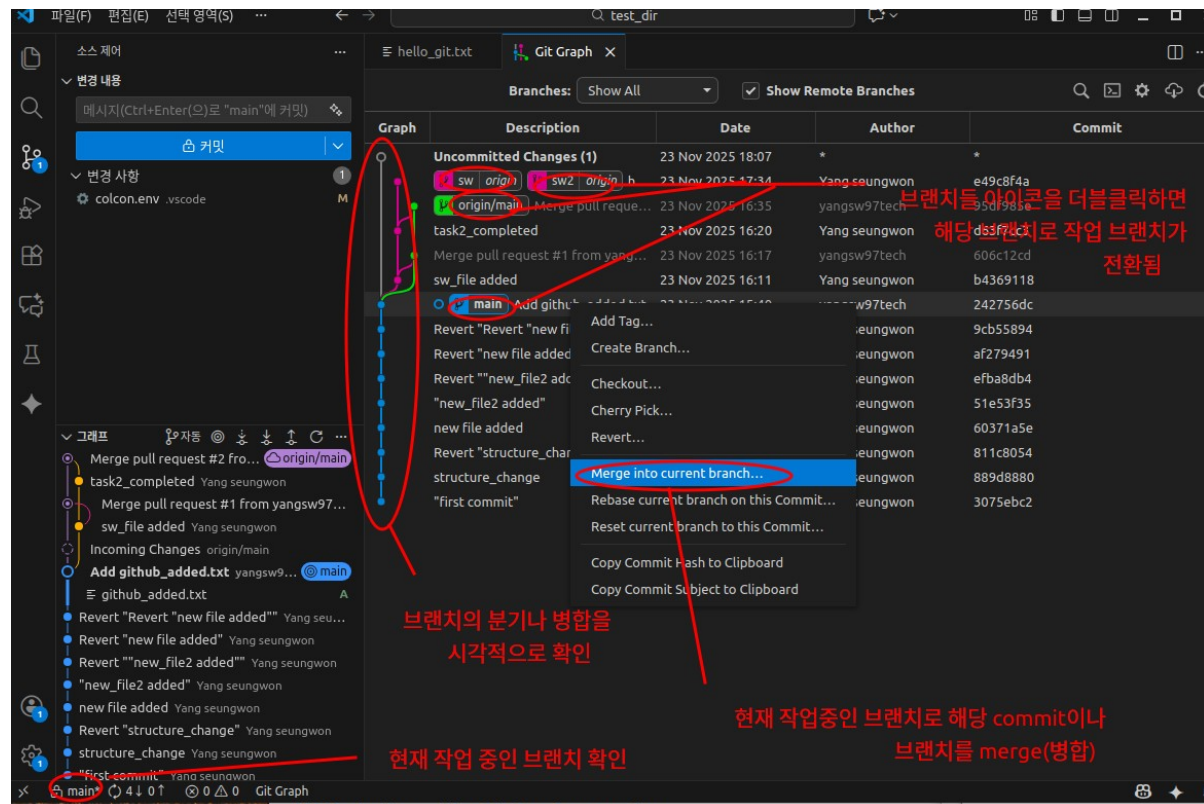
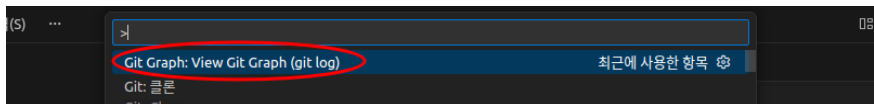
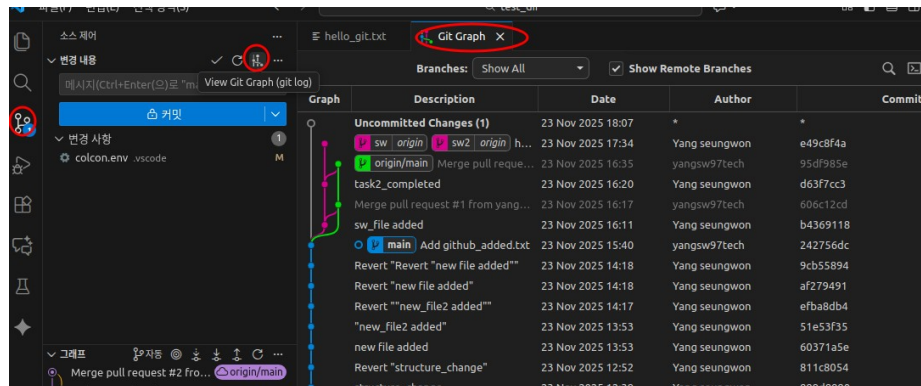
이를 통해 branch(분기)를 새로 생성하거나 현재 작업 branch 를 변경할 수도 있습니다. 보통 처음 자신의 분기를 생성하면 로컬 branch(origin/ 이 안붙은 branch)만 생성될 텐데, 여기서 한번 push 해주면 자동으로 원격 branch 가 생성됩니다.

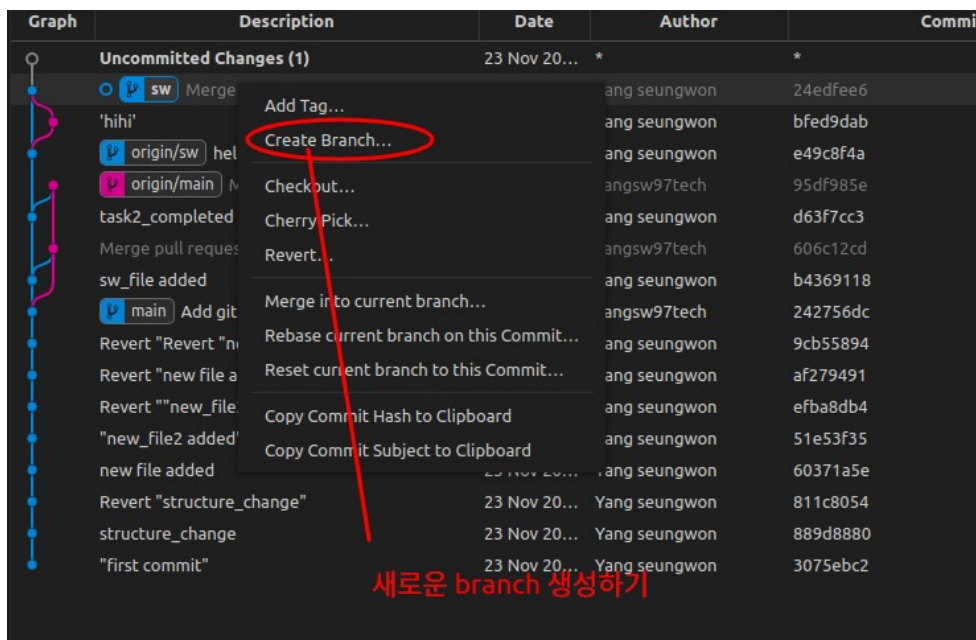
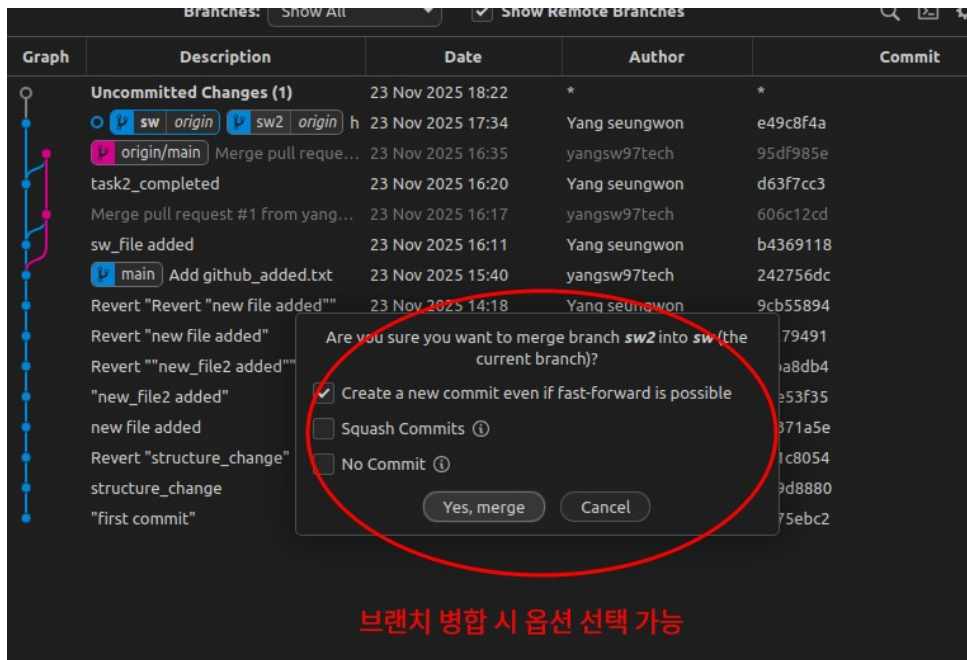


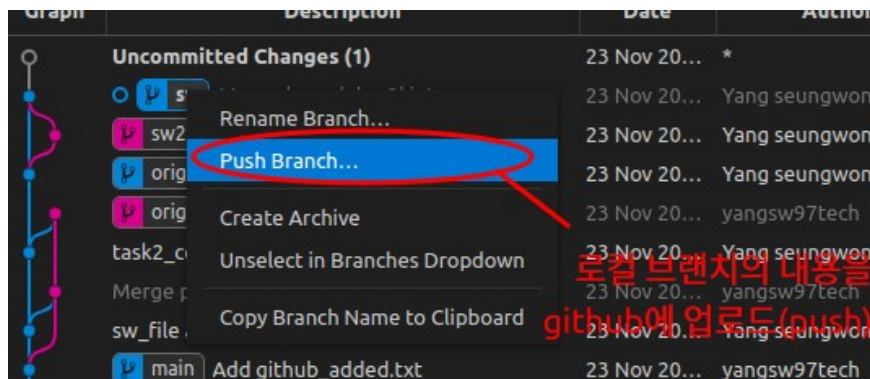
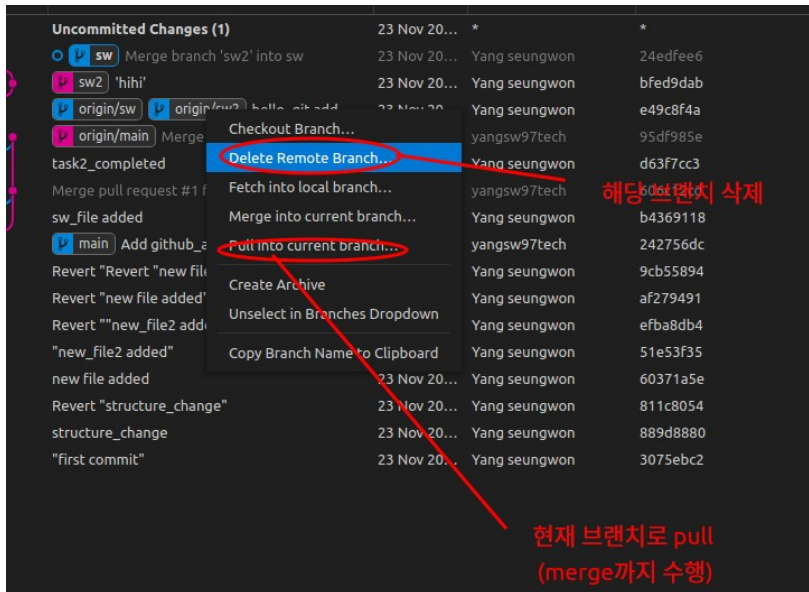
병합은 위와 같이 분기에서 병합을 선택할 수 있습니다.

Git graph 활용(권장)

이보다 더 좋은 활용 방법은 이전에 언급한 git graph 확장 기능을 이용하는 것입니다. 확장 설치 이후 아래와 같이 소스제어에서 해당 아이콘으로 git graph 를 키던가 'ctrl+shift+P'를 이용하여 명령 팔레트를 열고 git graph: view git graph (git log)를 입력하여 git graph 를 켤 수 있습니다.



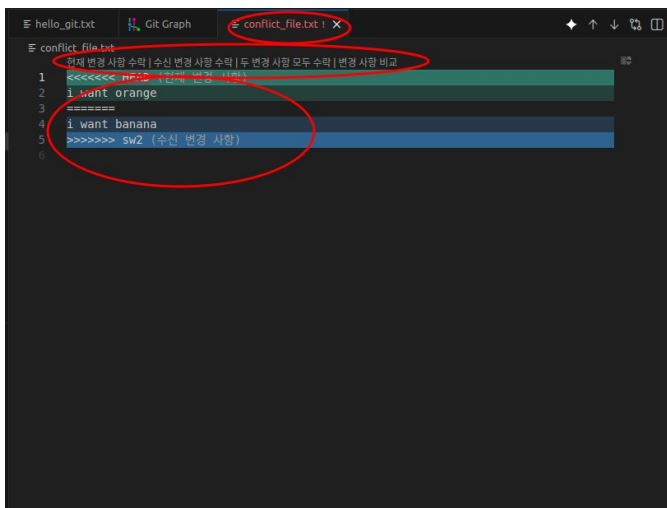
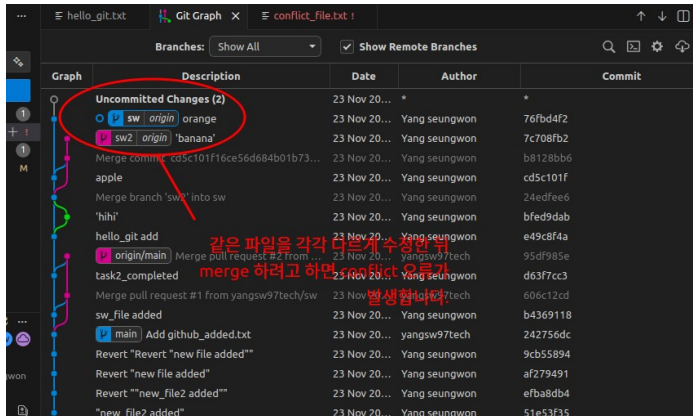




위 branch 이름중에 origin 이 붙은 것은 원격 저장소, 안붙은 것은 로컬 저장소에 해당하는 branch 라고 생각하면 됩니다.

아래와 같이 같은 파일을 서로 다르게 수정한 뒤 merge 하는 경우 conflict 가 발생합니다.

한 branch 는 banana 로, 다른 branch 는 orange 로 수정한 뒤 merge 하려는 경우 conflict 해결창이 뜨며 수정할 수 있게 됩니다. Vs code 의 버튼(현재 변경 사항 수락 / 수신 변경 사항 수락 / ...)을 통해서 수정할 수도 있고 직접 자신이 남길 내용만 작성하며 수정할 수도 있습니다.

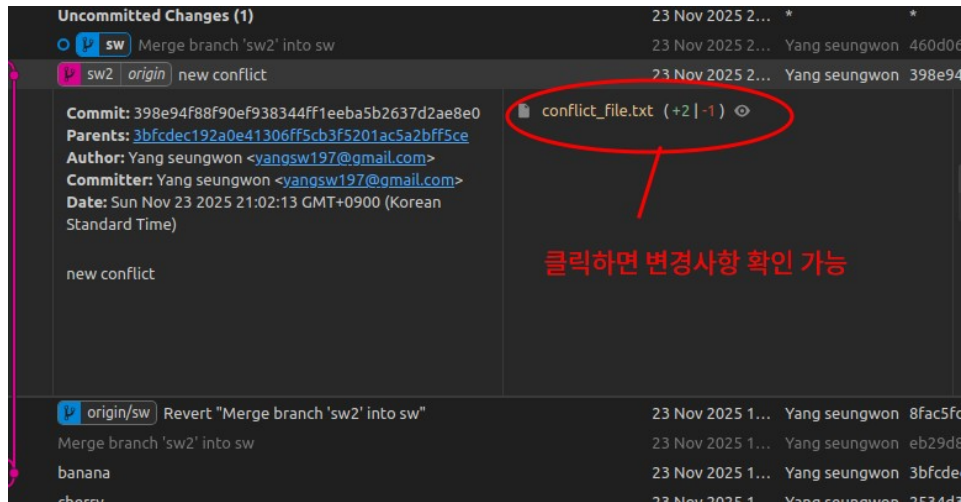


Conflict 파일 수정 후 소스제어에서 staging 하고 '계속' 버튼을 누르면 정상적으로 merge 가 되었음을 확인할 수 있습니다.

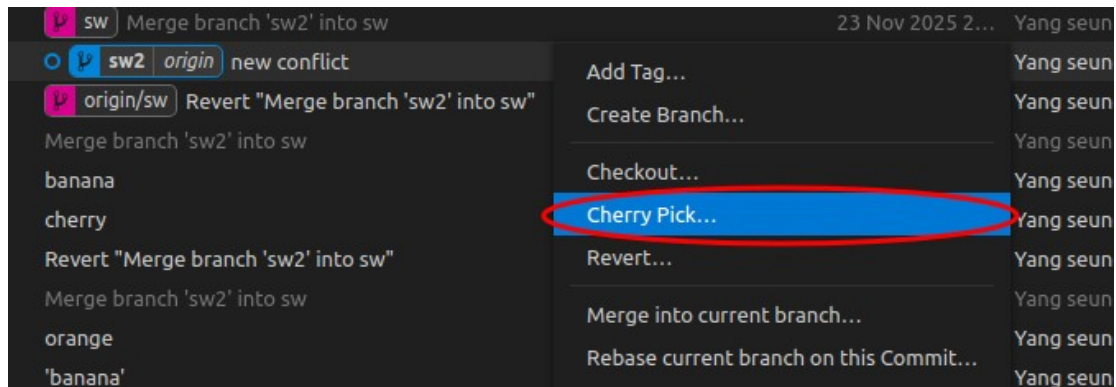
Merge 이외에도 git rebase 라는 병합 방식이 있는데, 이는 git merge 와 다르게 별도의 merge commit 이 생성되지 않고 선형적 히스토리가 생성되어 좀 더 깔끔한 히스토리 관리가 가능하므로 고려해볼 수 있는 방식입니다.

추가

git diff 명령어를 사용하여 두 commit 해시 간 혹은 두 branch 간 변경사항 비교가 가능하지만, git graph 로 변경되는 파일들에 대한 기록을 바로 확인할 수 있습니다.



Git graph 의 cherry pick 은 특정 commit 하나만 현재 브랜치에 적용시키려 할 때 사용합니다.



github 에서 commit history 를 확인하거나 파일 추가, 브랜치 확인 등 다양한 작업이 가능합니다.

The screenshot shows the GitHub repository interface for a user named 'test'. The 'Issues' tab is selected in the top navigation bar. A red circle highlights the 'Issues' tab, with a red line pointing to the text '코드 사용 중 일어난 오류나 버그에 대해 토의 가능' (Possible to discuss errors or bugs that occur during code use). Another red circle highlights the 'main' branch dropdown, with a red line pointing to the text 'branch를 변경하여 파일 확인 가능' (Possible to check files by changing the branch). A third red circle highlights the 'Add file' button, with a red line pointing to the text '직접 파일을 추가할 수 있음' (Possible to add files directly). A fourth red circle highlights the '13 Commits' link, with a red line pointing to the text 'commit 히스토리를 확인할 수 있음' (Possible to check commit history). The repository content shows a list of files and their commit history, including .vscode, hi, .gitignore, github_added.txt, hello_git.txt, and new_file.py.

The screenshot shows the 'Commits' page for the same repository. The 'main' branch is selected in the dropdown. A red circle highlights the 'main' branch dropdown, with a red line pointing to the text '브랜치 선택' (Branch selection). Another red circle highlights the 'All users' and 'All time' filters, with a red line pointing to the text 'user 또는 날짜별 조회 가능' (Possible to search by user or date). A third red circle highlights the commit hash '95df985', with a red line pointing to the text '해시 확인' (Check hash). A fourth red circle highlights the commit details, with a red line pointing to the text '해당 commit에서의 파일 구조로 확인' (Check file structure in the corresponding commit). The commit list shows several commits, including 'Merge pull request #2 from yangsw97tech/sw', 'task2_completed', 'Merge pull request #1 from yangsw97tech/sw', 'sw_file added', and 'Add github_added.txt'.